

torch.func.jacrev#

<https://pytorch.org/docs/stable/generated/torch.func.jacrev.html>

Description:

Computes the Jacobian of `func` with respect to the `arg(s)` at index `argnum` using reverse mode autodiff. Using `chunk_size=1` is equivalent to computing the jacobian row-by-row with a for-loop i.e. the constraints of `vmap()` are not applicable. `func` (function) – A Python function that takes one or more arguments, one of which must be a Tensor, and returns one or more Tensors. `argnums` (int or Tuple[int]) – Optional, integer or tuple of integers, saying which arguments to get the Jacobian with respect to. Default: 0.

Function Signature

```
torch.func.jacrev(func, argnums=0, *, has_aux=False,
chunk_size=None, _preallocate_and_copy=False)[source]#
```

Parameters

Returns

Returns:

Returns a function that takes in the same inputs as `func` and returns the Jacobian of `func` with respect to the `arg(s)` at `argnums`. If `has_aux` is `True`, then the returned function instead returns a `(jacobian, aux)` tuple where `jacobian` is the Jacobian and `aux` is auxiliary objects returned by `func`.

Code Examples

```
>>> from torch.func import jacrev
>>> x = torch.randn(5)
>>> jacobian = jacrev(torch.sin)(x)
>>> expected = torch.diag(torch.cos(x))
>>> assert torch.allclose(jacobian, expected)
```

```
>>> from torch.func import jacrev
>>> x = torch.randn(5)
>>>
>>> def f(x):
>>>     return x.sin()
>>>
>>> def g(x):
>>>     result = f(x)
>>>     return result, result
>>>
>>> jacobian_f, f_x = jacrev(g, has_aux=True)(x)
>>> assert torch.allclose(f_x, f(x))
```

```
>>> from torch.func import jacrev, vmap
>>> x = torch.randn(64, 5)
>>> jacobian = vmap(jacrev(torch.sin))(x)
>>> assert jacobian.shape == (64, 5, 5)
```

```
>>> from torch.func import jacrev
>>> def f(x):
>>>     return x.sin().sum()
>>>
>>> x = torch.randn(5)
>>> hessian = jacrev(jacrev(f))(x)
>>> assert torch.allclose(hessian, torch.diag(-x.sin()))
```

```
>>> from torch.func import jacrev
>>> def f(x, y):
>>>     return x + y ** 2
>>>
>>> x, y = torch.randn(5), torch.randn(5)
>>> jacobian = jacrev(f, argnums=1)(x, y)
>>> expected = torch.diag(2 * y)
>>> assert torch.allclose(jacobian, expected)
```

```
>>> from torch.func import jacrev
>>> def f(x, y):
>>>     return x + y ** 2
>>>
>>> x, y = torch.randn(5), torch.randn(5)
>>> jacobian = jacrev(f, argnums=(0, 1))(x, y)
>>> expectedX = torch.diag(torch.ones_like(x))
>>> expectedY = torch.diag(2 * y)
>>> assert torch.allclose(jacobian[0], expectedX)
>>> assert torch.allclose(jacobian[1], expectedY)
```

```
>>> def f(x):
>>>     with torch.no_grad():
>>>         c = x ** 2
>>>     return x - c
```

```
>>> with torch.no_grad():
>>>     jacrev(f)(x)
```

Notes & Warnings

NOTE

Note Using `chunk_size=1` is equivalent to computing the jacobian row-by-row with a for-loop i.e. the constraints of `vmap()` are not applicable.

NOTE

Note Using PyTorch `torch.no_grad` together with `jacrev`. Case 1: Using `torch.no_grad` inside a function:

```
>>> def f(x): >>> with torch.no_grad(): >>> c = x ** 2 >>> return x - c
```

 In this case, `jacrev(f)(x)` will respect the inner `torch.no_grad`. Case 2: Using `jacrev` inside `torch.no_grad` context manager:

```
>>> with torch.no_grad(): >>> jacrev(f)(x)
```

 In this case, `jacrev` will respect the inner `torch.no_grad`, but not the outer one. This is because `jacrev` is a “function transform”: its result should not depend on the result of a context manager outside of `f`.

Detailed Documentation

Documentation

Computes the Jacobian of `func` with respect to the `arg(s)` at index `argnum` using reverse mode autodiff

Using `chunk_size=1` is equivalent to computing the jacobian row-by-row with a for-loop i.e. the constraints of `vmap()` are not applicable.

`func` (function) – A Python function that takes one or more arguments, one of which

must be a Tensor, and returns one or more Tensors

`argnums` (int or Tuple[int]) – Optional, integer or tuple of integers, saying which arguments to get the Jacobian with respect to. Default: 0.

`has_aux` (bool) – Flag indicating that `func` returns a (output, aux) tuple where the first element is the output of the function to be differentiated and the second element is auxiliary objects that will not be differentiated. Default: False.

`chunk_size` (None or int) – If None (default), use the maximum chunk size (equivalent to doing a single `vmap` over `vjp` to compute the jacobian). If 1, then compute the jacobian row-by-row with a for-loop. If not None, then compute the jacobian `chunk_size` rows at a time (equivalent to doing multiple `vmap` over `vjp`). If you run into memory issues computing the jacobian, please try to specify a non-None `chunk_size`.

Returns a function that takes in the same inputs as `func` and returns the Jacobian of `func` with respect to the arg(s) at `argnums`. If `has_aux` is True, then the returned function instead returns a (jacobian, aux) tuple where jacobian is the Jacobian and aux is auxiliary objects returned by `func`.

A basic usage with a pointwise, unary operation will give a diagonal array as the Jacobian

If you would like to compute the output of the function as well as the jacobian of the function, use the `has_aux` flag to return the output as an auxiliary object:

`jacrev()` can be composed with `vmap` to produce batched Jacobians:

Additionally, `jacrev()` can be composed with itself to produce Hessians

By default, `jacrev()` computes the Jacobian with respect to the first input. However, it can compute the Jacobian with respect to a different argument by using `argnums`:

Additionally, passing a tuple to `argnums` will compute the Jacobian with respect to multiple arguments

Using PyTorch `torch.no_grad` together with `jacrev`. Case 1: Using `torch.no_grad` inside a function:

In this case, `jacrev(f)(x)` will respect the inner `torch.no_grad`.

Case 2: Using `jacrev` inside `torch.no_grad` context manager:

In this case, `jacrev` will respect the inner `torch.no_grad`, but not the outer one. This is because `jacrev` is a “function transform”: its result should not depend on the result of a context manager outside of `f`.